

RR: Round Robin

Implemented rules and examples for Scheduling Studio. These describe the current visualizer, including its deterministic tie and boundary ordering.

Model and terminology

The visualizer uses one CPU, discrete ticks, and CPU-only processes. A process has an arrival time and a service time. Service is the total CPU work required to finish; remaining service is the work still needed. Waiting does not consume service. There is no I/O, blocking, or context-switch cost in the normal UI.

A snapshot at time t shows the state after the scheduling decisions at t and before execution during $[t, t + 1)$. If a process runs during that interval, its remaining service decreases by one. Completion is recorded at the following boundary. Finished processes remain in the metrics table but leave the CPU and ready queue.

1. Use one FIFO ready queue

Arrivals join the back. Simultaneous arrivals retain input order. When the CPU is free, dispatch the queue head. Service length and process ID do not affect selection.

Example: Ready order C, A, B dispatches C first, even if B has the shortest service.

2. Give each turn a fresh quantum

Quantum and time slice mean the same thing: the maximum CPU ticks allowed in one turn. The UI defaults to quantum 2. Each running tick consumes one service tick and one quantum tick. Waiting consumes neither. A process can finish before using its whole quantum.

The quantum must be a positive safe whole number. A valid quantum larger than the entire workload is allowed. RR has no separate allotment, priority levels, demotion, or boost. MLFQ settings do not affect it.

Example: A needs five service ticks and has quantum 2. After its first two-tick turn, it has three service ticks remaining. Its next turn starts with a fresh two-tick quantum.

3. Arrivals do not interrupt an ongoing turn

A newly arriving process waits at the back until the current process finishes or exhausts its quantum. A shorter service requirement gives it no special priority.

Example: A runs from 0 with quantum 3. B arrives at 1. If A is unfinished, it keeps running through 2–3 and its turn ends at 3.

4. Completion wins over quantum expiry

At the next boundary, remove a runner whose remaining service has reached zero. Do not rotate it, even if its quantum was used exactly.

Example: A needs two ticks and quantum is 2. It finishes at 2 and leaves; there is no extra queue visit or new turn.

5. Rotate unfinished work to the back

If the runner has used its quantum and is unfinished, reset its turn usage to zero and append it to the ready-queue tail. Preserve remaining service and its accumulated metrics.

Example: With quantum 2, A has three service ticks left when its turn expires and B is waiting. Queue order becomes B, A; B runs next.

6. Admit same-time arrivals before the expired runner

At each boundary:

1. Remove a completed runner.
2. Detect quantum expiry for an unfinished runner and hold it for requeueing.
3. Append all arrivals at this time, in input order.
4. Append the expired runner with a fresh quantum, if one exists.
5. Dispatch the queue head if the CPU is free; otherwise continue the ongoing turn.
6. Record the boundary, then run the next tick unless all processes have finished.

Existing waiting processes stay ahead of the new arrivals, and those arrivals stay ahead of the expired runner.

Example: Quantum is 2. A arrives at zero with four service ticks; B arrives at 2 with one. At 2, B enters before A is requeued, so B runs during 2–3. The complete sequence is A A B A A.

7. Handle single processes, small quanta, and idle periods

A lone unfinished process still expires, rejoins the queue, and can immediately be selected again at the same boundary. That costs no simulated tick. Quantum 1 rotates every running tick unless the process finishes. If the quantum is at least every process's service, each finishes in one turn and the execution order is FCFS.

The CPU is idle only when no process is ready. Future arrivals remain outside the ready queue until their arrival time. Stop after all processes finish.

Example: One process with five service ticks and quantum 2 runs continuously during 0–5, with renewals at 2 and 4.

Worked example

The following independent example uses input order A, B, C:

Process	Arrival	Service
A	0	5
B	1	2
C	1	1

Use quantum 2.

Interval	CPU	Reason
0–2	A	A uses its first quantum; B and C arrive at 1.
2–4	B	B is the head after A rotates; it finishes exactly at expiry.
4–5	C	C finishes before using its full quantum.
5–7	A	A resumes for two ticks and has one service tick left.
7–8	A	Alone in the queue, A is immediately redispached with a fresh quantum.

Per-tick CPU sequence: A A B B C A A A. At time 8, all processes are finished.

Process	Response	Waiting	Turnaround
A	0	3	8
B	1	1	3
C	3	3	4

Metrics and display

- **Response time:** first CPU start minus arrival time. It is recorded once and remains unchanged afterward.
- **Waiting time:** total ticks spent ready but not running. Time before arrival is excluded. At completion, waiting equals turnaround minus original service.
- **Turnaround time:** completion time minus arrival time. It remains unavailable until the process finishes.
- **Averages:** waiting includes every listed process, including future processes with zero waiting. Response averages include only processes that have started; turnaround averages include only finished processes. An unavailable value is shown as a dash. Non-integer averages are displayed to one decimal place.
- **Cards and timeline:** `N service left` is remaining CPU work. A timeline cell at `t` identifies the process running during `[t, t + 1)`, or idle. Animation and reverse playback visualize the same recorded schedule and do not change its decisions.

Input limits and recovery

Use 1–50 processes. IDs must be nonempty and unique after trimming whitespace and ignoring case. Arrival times must be nonnegative safe whole numbers; service times must be positive safe whole numbers. The complete workload, including idle gaps, must finish by tick 2,000. Finishing exactly at 2,000 is allowed; finishing at 2,001 is rejected.

The numeric editor floors fractional entries and clamps them to the field minimum. JSON imports and direct engine calls must pass integer validation. Invalid edits replace the visualization with a validation message; an invalid JSON import leaves the current scenario unchanged. Editing a valid scenario or changing the policy restarts playback at time zero.